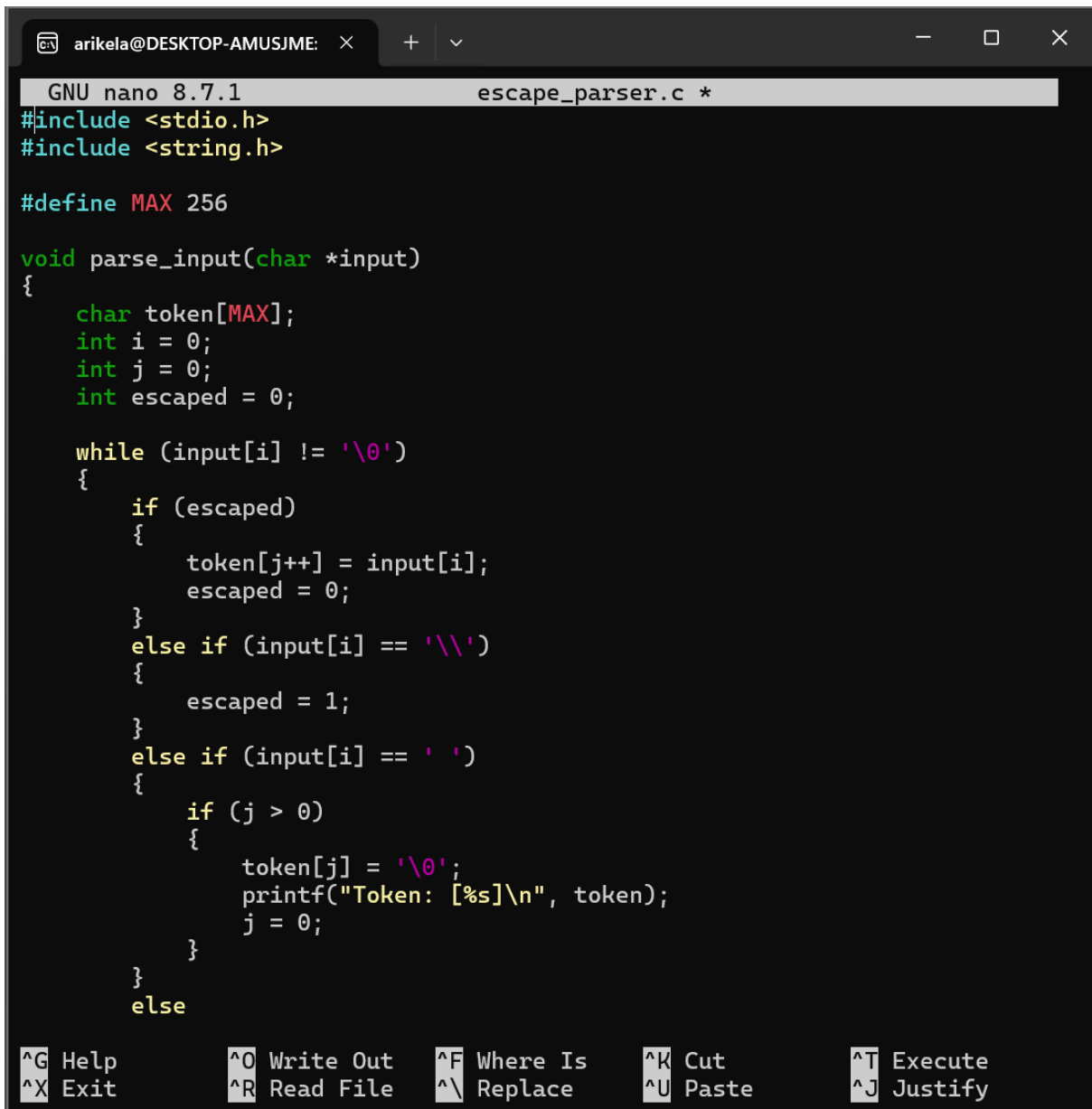


Skill-Week-6

Task-1: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs



```
GNU nano 8.7.1 escape_parser.c *
#include <stdio.h>
#include <string.h>

#define MAX 256

void parse_input(char *input)
{
    char token[MAX];
    int i = 0;
    int j = 0;
    int escaped = 0;

    while (input[i] != '\0')
    {
        if (escaped)
        {
            token[j++] = input[i];
            escaped = 0;
        }
        else if (input[i] == '\\')
        {
            escaped = 1;
        }
        else if (input[i] == ' ')
        {
            if (j > 0)
            {
                token[j] = '\0';
                printf("Token: [%s]\n", token);
                j = 0;
            }
        }
        else
        {
            token[j++] = input[i];
        }
    }
}
```

^G Help **^O** Write Out **^F** Where Is **^K** Cut **^T** Execute
^X Exit **^R** Read File **^_** Replace **^U** Paste **^J** Justify

```

        else if (input[i] == ' ')
        {
            if (j > 0)
            {
                token[j] = '\\0';
                printf("Token: [%s]\\n", token);
                j = 0;
            }
        }
        else
        {
            token[j++] = input[i];
        }

        i++;
    }

    if (escaped)
    {
        token[j++] = '\\\\';
    }

    if (j > 0)
    {
        token[j] = '\\0';
        printf("Token: [%s]\\n", token);
    }
}

```

```

int main()
{

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^\\ Replace	^U Paste	^J Justify

```

int main()
{
    char input[MAX];

    printf("Enter input: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\\n")] = '\\0';

    printf("\\nParsing result:\\n");
    parse_input(input);

    return 0;
}

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^\\ Replace	^U Paste	^J Justify

```
arikel@DESKTOP-AMUSJME:~$ mkdir week5skill
arikel@DESKTOP-AMUSJME:~$ cd week5skill
arikel@DESKTOP-AMUSJME:~/week5skill$ nano escape_parser.c
arikel@DESKTOP-AMUSJME:~/week5skill$ gcc escape_parser.c -o escape_parser
arikel@DESKTOP-AMUSJME:~/week5skill$ ./escape_parser
Enter input: hello world

Parsing result:
Token: [hello]
Token: [world]
arikel@DESKTOP-AMUSJME:~/week5skill$ ./escape_parser
Enter input: hello\ world

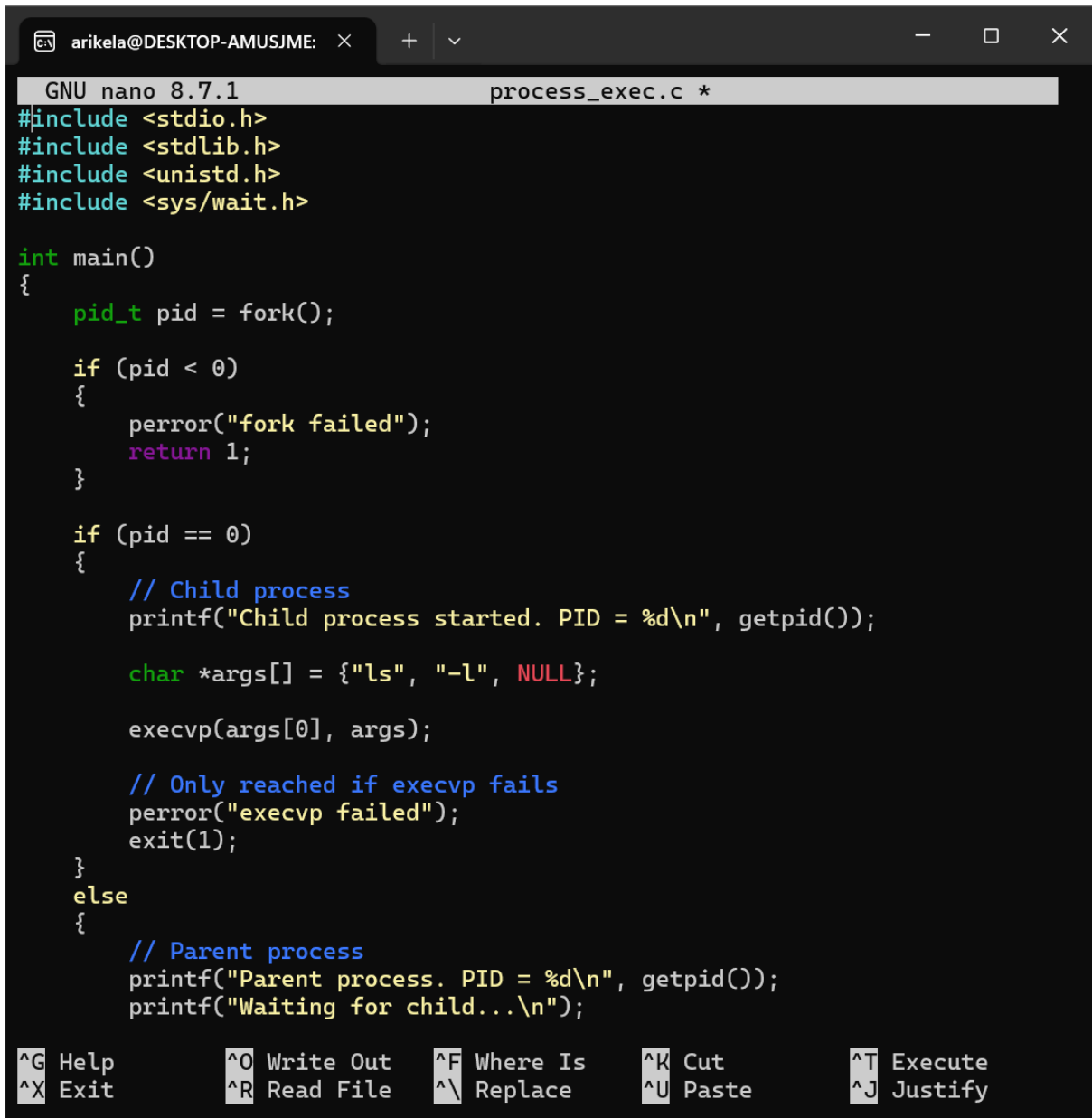
Parsing result:
Token: [hello world]
arikel@DESKTOP-AMUSJME:~/week5skill$ ./escape_parser
Enter input: hello\@world

Parsing result:
Token: [hello@world]
arikel@DESKTOP-AMUSJME:~/week5skill$ ./escape_parser
Enter input: hello\ big\ world

Parsing result:
Token: [hello big world]
arikel@DESKTOP-AMUSJME:~/week5skill$ ./escape_parser
Enter input: my\ file.txt hello\ world test

Parsing result:
Token: [my file.txt]
Token: [hello world]
Token: [test]
arikel@DESKTOP-AMUSJME:~/week5skill$
```

Task-2: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching lets complete these tasks in the same way



```
GNU nano 8.7.1 process_exec.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        // Child process
        printf("Child process started. PID = %d\n", getpid());

        char *args[] = {"ls", "-l", NULL};

        execvp(args[0], args);

        // Only reached if execvp fails
        perror("execvp failed");
        exit(1);
    }
    else
    {
        // Parent process
        printf("Parent process. PID = %d\n", getpid());
        printf("Waiting for child...\n");
    }
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify

```

else
{
    // Parent process
    printf("Parent process. PID = %d\n", getpid());
    printf("Waiting for child...\n");

    wait(NULL);

    printf("Child process completed.\n");
}

return 0;
}

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

```

arikel@DESKTOP-AMUSJME:~/week5skill$ nano process_exec.c
arikel@DESKTOP-AMUSJME:~/week5skill$ gcc process_exec.c -o process_exec
arikel@DESKTOP-AMUSJME:~/week5skill$ ./process_exec
Parent process. PID = 763
Waiting for child...
Child process started. PID = 764
total 40
-rwxr-xr-x 1 arikela arikela 16216 Sep 17 09:15 escape_parser
-rw-r--r-- 1 arikela arikela 1051 Sep 17 09:15 escape_parser.c
-rwxr-xr-x 1 arikela arikela 16304 Sep 17 09:22 process_exec
-rw-r--r-- 1 arikela arikela 714 Sep 17 09:22 process_exec.c
Child process completed.
arikel@DESKTOP-AMUSJME:~/week5skill$

```

Change the code and compile:

```

if (pid == 0)
{
    // Child process
    printf("Child process started. PID = %d\n", getpid());

    char *args[] = {"ls", "-a", NULL};

    execvp(args[0], args);

    // Only reached if execvp fails
    perror("execvp failed");
    exit(1);
}

```

```

arikel@DESKTOP-AMUSJME:~/week5skill$ nano process_exec.c
arikel@DESKTOP-AMUSJME:~/week5skill$ gcc process_exec.c -o process_exec
arikel@DESKTOP-AMUSJME:~/week5skill$ ./process_exec
Parent process. PID = 817
Waiting for child...
Child process started. PID = 818
. .. escape_parser escape_parser.c process_exec process_exec.c
Child process completed.
arikel@DESKTOP-AMUSJME:~/week5skill$ |

```

Change the code to test execution error handling:

```

if (pid == 0)
{
    // Child process
    printf("Child process started. PID = %d\n", getpid());

    char *args[] = {"doesnotexist", NULL};

    execvp(args[0], args);

    // Only reached if execvp fails
    perror("execvp failed");
    exit(1);
}
else

```

```

arikela@DESKTOP-AMUSJME:~/week5skill$ nano process_exec.c
arikela@DESKTOP-AMUSJME:~/week5skill$ gcc process_exec.c -o process_exec
arikela@DESKTOP-AMUSJME:~/week5skill$ ./process_exec
Parent process. PID = 858
Waiting for child...
Child process started. PID = 859
execvp failed: No such file or directory
Child process completed.

```

Now change the code back to normal(correct code):

```

arikela@DESKTOP-AMUSJME:~/week5skill$ nano process_exec.c
arikela@DESKTOP-AMUSJME:~/week5skill$ gcc process_exec.c -o process_exec
arikela@DESKTOP-AMUSJME:~/week5skill$ ./process_exec
Parent process. PID = 901
Waiting for child...
Child process started. PID = 902
total 40
-rwxr-xr-x 1 arikela arikela 16216 Sep 17 09:15 escape_parser
-rw-r--r-- 1 arikela arikela 1051 Sep 17 09:15 escape_parser.c
-rwxr-xr-x 1 arikela arikela 16304 Sep 17 09:32 process_exec
-rw-r--r-- 1 arikela arikela 713 Sep 17 09:31 process_exec.c
Child process completed.
arikela@DESKTOP-AMUSJME:~/week5skill$ |

```